Supplementary material

When Does SHAP Outperform LIME? Model- and Configuration-Dependent Results from a Paired Tabular Benchmark. Manuscript 2067810, *Computers and Informatics*, revision 1.

This file is independent of the paper: it states the whole design and every result at the level of dataset, model and seed, so that each number in the paper can be checked. Repository and archive addresses are withheld for masked review.

Design fingerprint (SHA-256 of the design configuration): `4ab8673a4a205aaf70ac514a83a21abb59c9ff951a111a6679a85dfe4d97a1fa`. Every result row carries it; the analysis refuses rows of another design.

S1. Design

Item	Setting
Split	one stratified split per dataset, test share 0.2, seed 42; shared by preprocessing, training and every seed
Training cap	20,000 rows (stratified subsample of the training split)
Seeds	Adult: (11, 22, 33, 44, 55); other datasets: (11, 22, 33)
Instances	up to 25 per stratum (TP, TN, FP, FN) of the test split; smaller strata taken whole
Stability subset	first 3 instances per stratum
Stability	T = 15 perturbed copies, Gaussian noise sigma = 0.1 on the encoded features
Faithfulness gap	top k = 5 features masked together
Sparsity threshold	0.0001
Timing subset	5 instances per stratum, 3 repetitions, 1 warm-up per explainer, one process, one thread, pinned to one core

S2. Datasets

Dataset	Source	Train rows	Test rows	Encoded features	Positive share	Role
Adult	UCI	39,032	9,758	108	0.239	primary
Breast Cancer	UCI	455	114	30	0.627	further
German Credit	UCI	800	200	61	0.700	further
Diabete	OpenM	614	154	8	0.349	numeric, lower

Dataset	Source	Train rows	Test rows	Encoded features	Positive share	Role
s	L 37					dimension
Spambase	OpenML 44	3,680	921	57	0.394	numeric, higher dimension
Phishing	OpenML 4534	8,844	2,211	68	0.557	categorical, lower dimension
Connect-4	OpenML 40668	54,045	13,512	126	0.658	categorical, higher dimension

Train rows are those available before the training cap. Connect-4: a win is the positive class; loss and draw are the negative class.

S3. Models

Model	Class	Hyperparameters
Logistic regression	`sklearn.linear_model.LogisticRegression`	solver=lbfgs, max_iter=1000, random_state=42
Random forest	`sklearn.ensemble.RandomForestClassifier`	n_estimators=100, max_depth=15, random_state=42, n_jobs=1
XGBoost	`xgboost.XGBClassifier`	n_estimators=100, max_depth=6, learning_rate=0.1, objective=binary:logistic, eval_metric=logloss, random_state=42, n_jobs=1
SVM (RBF)	`sklearn.svm.SVC`	C=1.0, kernel=rbf, probability=True, random_state=42
MLP	`sklearn.neural_network.MLPClassifier`	hidden_layer_sizes=(100), max_iter=500, random_state=42

Test accuracy of the frozen models:

Dataset	Logistic regression	Random forest	XGBoost	SVM (RBF)	MLP
Adult	0.8500	0.8580	0.8666	0.8526	0.8240
Breast Cancer	0.9825	0.9561	0.9474	0.9825	0.9649
German Credit	0.7050	0.7400	0.7400	0.7400	0.7400
Diabetes	0.7143	0.7532	0.7662	0.7532	0.7078
Spambase	0.9294	0.9425	0.9468	0.9273	0.9392
Phishing	0.9398	0.9715	0.9665	0.9593	0.9742

Dataset	Logistic regression	Random forest	XGBoost	SVM (RBF)	MLP
Connect-4	0.7979	0.8290	0.8529	0.8490	0.8676

S4. Explainers

Variant	Settings
shap	method=shap, n_background_samples=50, kernel_nsamples=500
lime	method=lime, num_samples=1000, num_features=10, kernel_width=default
lime_full	method=lime, num_samples=1000, num_features=all, kernel_width=default
anchors	method=anchors, threshold=0.95
dice	method=dice, num_counterfactuals=1

SHAP uses TreeSHAP for the random forest and XGBoost and KernelSHAP otherwise. LIME's default kernel width is 0.75 times the square root of the number of encoded features. Anchors: the attribution vector marks the anchor's features with 1. DiCE: the attribution of a feature is the absolute change in the counterfactual.

S5. Computing environment

One laptop for every run: AMD Ryzen 5 4600H (6 cores, 12 threads, 3.0 GHz base), 31.4 GB RAM, Windows 11 Home 10.0.26300, on mains power. CPython 3.11.11; scikit-learn 1.7.1, XGBoost 3.1.2, SHAP 0.50.0, LIME 0.2.0.1, NumPy 2.2.6, pandas 2.3.3, SciPy 1.16.3, alibi 0.9.6, dice-ml 0.12. The full frozen environment is part of the artifacts.

S6. Model and data fingerprints

Each run checks the model file against these hashes before it starts.

Dataset	Model	Model SHA-256 (first 16)	Data SHA-256 (first 16)
Adult	Logistic regression	`c06174920a8dfa75`	`18b42fcbbcf14068`
Adult	MLP	`329f273fa5d5688a`	`18b42fcbbcf14068`
Adult	Random forest	`a2dd7551756486ea`	`18b42fcbbcf14068`
Adult	SVM (RBF)	`2b0a0c6b236bd11d`	`18b42fcbbcf14068`
Adult	XGBoost	`6fa7994e30549e68`	`18b42fcbbcf14068`
Breast Cancer	Logistic regression	`571aaf8bb37da91f`	`765348e4109f0b13`
Breast Cancer	MLP	`9e7838f9c2a4616e`	`765348e4109f0b13`
Breast Cancer	Random forest	`ece4da48e4fcc84d`	`765348e4109f0b13`
Breast	SVM (RBF)	`273b476e04b1f1dd`	`765348e4109f0b13`

Dataset	Model	Model SHA-256 (first 16)	Data SHA-256 (first 16)
Cancer			
Breast Cancer	XGBoost	`1d03c4d9da6ab8a9`	`765348e4109f0b13`
Connect-4	Logistic regression	`873e0b72df24adfc`	`e438bafb45e36a75`
Connect-4	MLP	`d737986b44181bc3`	`e438bafb45e36a75`
Connect-4	Random forest	`213c98dad25a4fc7`	`e438bafb45e36a75`
Connect-4	SVM (RBF)	`0cbc54c363e606c5`	`e438bafb45e36a75`
Connect-4	XGBoost	`6d0ea32aa008aa18`	`e438bafb45e36a75`
Diabetes	Logistic regression	`a991171cff6e2c47`	`f9f1195b4e6a9677`
Diabetes	MLP	`6e2d11ebedfcff6c`	`f9f1195b4e6a9677`
Diabetes	Random forest	`006d37382f8dfe45`	`f9f1195b4e6a9677`
Diabetes	SVM (RBF)	`82d2e966fea86ace`	`f9f1195b4e6a9677`
Diabetes	XGBoost	`51545d5ae1130e5a`	`f9f1195b4e6a9677`
German Credit	Logistic regression	`f94432191bc002cf`	`04b09e7ae7547830`
German Credit	MLP	`4c406969284f6c6c`	`04b09e7ae7547830`
German Credit	Random forest	`fdbf9f1e3859d9c7`	`04b09e7ae7547830`
German Credit	SVM (RBF)	`0814b148809bf258`	`04b09e7ae7547830`
German Credit	XGBoost	`15ed0d95627e82d3`	`04b09e7ae7547830`
Phishing	Logistic regression	`b691431585152391`	`9d93cc662c21ea41`
Phishing	MLP	`11980a4dcd44ab8e`	`9d93cc662c21ea41`
Phishing	Random forest	`552a74bafd59aa88`	`9d93cc662c21ea41`
Phishing	SVM (RBF)	`b5ef782a7937d841`	`9d93cc662c21ea41`
Phishing	XGBoost	`c3583f6aef8d7f67`	`9d93cc662c21ea41`
Spambase	Logistic regression	`df2e6e9f26ecdc5b`	`89f80a488462e60e`
Spambase	MLP	`ec3acdf8a11764f8`	`89f80a488462e60e`
Spambase	Random forest	`b788ed367089fd13`	`89f80a488462e60e`
Spambase	SVM (RBF)	`9c3450727360d61a`	`89f80a488462e60e`

Dataset	Model	Model SHA-256 (first 16)	Data SHA-256 (first 16)
Spambase	XGBoost	`0b808e596faf5ff0`	`89f80a488462e60e`

S7. Realized instances

[R] Instances per stratum for each dataset, model and seed; overlap of instances between seeds; all-zero vectors and undefined fidelity per explainer. Written by the analysis of cohort B2 from the result rows.

S8. Primary benchmark: per model and seed

[R] Per cell (model, seed): mean stability, fidelity, faithfulness gap and sparsity for the five explainer variants; distribution of the instance-level paired differences; mixed-model variance components; cluster-bootstrap intervals. Written by the analysis of cohort B2 from the result rows.

S9. Cost

[R] Per dataset and model: median CPU time, rows evaluated, repetitions, and the quiet-host record; FLOPs for logistic regression and the MLP; the random forest against XGBoost at matched size. Written by the analysis of cohort B2 from the result rows.

S10. Configuration sweep

[R] Grids of LIME kernel width and samples and of KernelSHAP budget and background; per-model Pareto fronts; share of degenerate explanations. Written by the analysis of cohort B2 from the result rows.

S11. Measurement controls

[R] No-noise repeats; valid one-hot perturbation; rank and top-k stability; exclusion of all-zero vectors; four masking schemes on every model. Written by the analysis of cohort B2 from the result rows.

S12. Further datasets: per model

[R] Stability, fidelity, faithfulness gap, sparsity and cost for every dataset, model and explainer, mean and SD over seeds. Written by the analysis of cohort B2 from the result rows.

S13. How to regenerate every table

1. Install the frozen environment; check the model hashes of S6 (`paper_b_run.py preflight`).
2. Quality pass: `paper_b_run.py launch` (resumable; one row per instance).
3. Cost pass: `paper_b_run.py launch-timing` on an idle host.

4. Analysis: the scripts named under each table read the result rows and print the tables of the paper and of this file. [R] names of the analysis scripts.